7.4.3 Breaking the law: a subtle bug

Let's try this mode of thinking. We're expecting that `fork(x) == x` for *all* choices of `x`, and any choice of `ExecutorService`. We have a good sense of what `x` could be—it's some expression making use of `fork`, `unit`, and `map2` (and other combinators derived from these). What about `ExecutorService`? What are some possible implementations of it? There's a good listing of different implementations in the class `java.util.concurrent.Executors` (API link: http://mng.bz/urQd).

EXERCISE 7.8

Hard: Take a look through the various static methods in `Executors` to get a feel for the different implementations of `ExecutorService` that exist. Then, before continuing, go back and revisit your implementation of `fork` and try to find a counterexample or convince yourself that the law holds for your implementation.

Why laws about code and proofs are important

It may seem unusual to state and prove properties about an API. This certainly isn't something typically done in ordinary programming. Why is it important in FP?

In functional programming it's easy, and expected, to factor out common functionality into generic, reusable components that can be *composed*. Side effects hurt compositionality, but more generally, any hidden or out-of-band assumption or behavior that prevents us from treating our components (be they functions or anything else) as *black boxes* makes composition difficult or impossible.

In our example of the law for `fork`, we can see that if the law we posited didn't hold, many of our general-purpose combinators, like `parMap`, would no longer be sound (and their usage might be dangerous, since they could, depending on the broader parallel computation they were used in, result in deadlocks).

Giving our APIs an algebra, with laws that are meaningful and aid reasoning, makes the APIs more usable for clients, but also means we can treat the objects of our APIs as black boxes. As we'll see in part 3, this is crucial for our ability to factor out common patterns across the different libraries we've written.

There's actually a rather subtle problem that will occur in most implementations of `fork`. When using an `ExecutorService` backed by a thread pool of bounded size (see `Executors.newFixedThreadPool`), it's very easy to run into a deadlock.[14] Suppose we have an `ExecutorService` backed by a thread pool where the maximum number of threads is 1. Try running the following example using our current implementation:

[14] In the next chapter, we'll write a combinator library for testing that can help discover problems like these automatically.

```
val a = lazyUnit(42 + 1)
val S = Executors.newFixedThreadPool(1)
println(Par.equal(S)(a, fork(a)))
```

Most implementations of `fork` will result in this code deadlocking. Can you see why? Let's look again at our implementation of `fork`:

```
def fork[A](a: => Par[A]): Par[A] =
  es => es.submit(new Callable[A] {
    def call = a(es).get
  })
```

Waits for the result of one `Callable` inside another `Callable`.

Note that we're submitting the `Callable` first, and *within that* `Callable`, we're submitting another `Callable` to the `ExecutorService` and blocking on its result (recall that `a(es)` will submit a `Callable` to the `ExecutorService` and get back a `Future`). This is a problem if our thread pool has size 1. The outer `Callable` gets submitted and picked up by the sole thread. Within that thread, before it will complete, we submit and block waiting for the result of another `Callable`. But there are no threads available to run this `Callable`. They're waiting on each other and therefore our code deadlocks.

EXERCISE 7.9

Hard: Show that any fixed-size thread pool can be made to deadlock given this implementation of `fork`.

When you find counterexamples like this, you have two choices—you can try to fix your implementation such that the law holds, or you can refine your law a bit, to state more explicitly the conditions under which it holds (you could simply stipulate that you require thread pools that can grow unbounded). Even this is a good exercise—it forces you to document invariants or assumptions that were previously implicit.

Can we fix `fork` to work on fixed-size thread pools? Let's look at a different implementation:

```
def fork[A](fa: => Par[A]): Par[A] =
  es => fa(es)
```

This certainly avoids deadlock. The only problem is that we aren't actually forking a separate logical thread to evaluate `fa`. So `fork(hugeComputation)(es)` for some `ExecutorService` `es`, would run `hugeComputation` in the main thread, which is exactly what we wanted to avoid by calling `fork`. This is still a useful combinator, though, since it lets us delay instantiation of a computation until it's actually needed. Let's give it a name, `delay`:

```
def delay[A](fa: => Par[A]): Par[A] =
  es => fa(es)
```